

Geoprojekt Random Forest ML



points_shi_height_temp_rain_4326.csv

Arbeitsblatt

ich muss für meine geoprojektarbeit mit random forest ein maschinelles lernen entwickelt für unser projekt in python. 1. RMSE, Varianz, Gruppentrennung

👉 Ziel im RF:

RMSE minimieren → Vorhersagefehler klein

Varianz innerhalb Gruppen klein

Unterschied zwischen Gruppen groß

✅ Formulierung:

Splits werden so gewählt, dass die Homogenität innerhalb der Gruppen maximiert und die Unterschiede zwischen den Gruppen maximiert werden. [3_GEOProj_rF | PDF]

✅ 2. ML soll SHI erklären

👉 Ja, genau euer Ziel:

SHI = f(Klima, Geologie, Nutzung, Topographie)

👉 Ergebnis:

welche Variablen wichtig sind

wie sie wirken

welche Kombinationen wichtig sind

✅ 3. Gleiche Anzahl Splitpoints

👉 Idee:

nicht immer die gleiche Variable splitten

alle Variablen sollen „Chance“ bekommen

👉 wird erreicht durch:

mtry (Random Feature Selection)

✅ verhindert Bias

✅ 4. Nicht zu viele Gruppen

👉 Problem:

zu viele Splits → Overfitting

👉 Lösung:

minsplitt, minbucket

Conditional Inference (p-Wert)

✓ 5. Splitpoint finden (z. B. Temperatur)

☞ RF testet automatisch:

Temp < 8?

Temp < 10?

Temp < 12?

☞ nimmt den Split:

➡ mit kleinstem Fehler (RMSE)

✓ 6. Conditional Inference Tree ($p < 0.001$)

☞ JA sehr wichtig bei euch ✓

☞ bedeutet:

Split nur wenn statistisch signifikant

✓ Vorteile:

weniger Overfitting

fairere Variable Selection

☞ genau wie im Bild ($p < 0.001$)

✓ 7. Bootstrapping

☞ RF macht:

zufällige Stichprobe (mit Zurücklegen)

→ neuer Baum

✓ Ergebnis:

unterschiedliche Verteilungen

robustes Modell

✓ 8. Wie viele Variablen pro Split?

☞ Parameter:

mtry

☞ Empfehlung:

$\sqrt{\text{Anzahl Variablen}}$

☞ z. B. bei 4 Variablen:

mtry = 2

✓ 9. Prediction & OOB

☞ Out-of-Bag:

Daten, die NICHT im Baum waren

☞ darauf testen:

- ✓ echte Modellgüte
- ✓ wie gut für neue Daten

✓ 10. Lat/Long

☞ korrekt:

- ✓ nicht direkt nutzen

Warum:

ML kann schlecht mit Koordinaten umgehen

☞ besser:

später räumlich visualisieren

✓ 11. Kovarianz (Korrelation)

☞ Problem:

Temp ~ Höhe (hoch korreliert)

Lösung:

☞ Schritt 1:

Korrelationsmatrix

☞ Schritt 2:

Wenn $|r| > 0.8$:

☞ Optionen:

eine Variable entfernen

oder später vorsichtig interpretieren

✓ 12. Data Mining

☞ nach Modell:

- ✓ ihr analysiert:

Feature Importance

Zusammenhänge

Interaktionen

Szenarien

✓ 13. Bias-Varianz

☞ Idee:

Fehler = Bias + Varianz

Ziel:

nicht zu einfach

nicht zu komplex



prüfen:

$$R^2 = 1 - (\text{MSE} / \text{Varianz})$$

👉 je höher → besser

✓ 14. Parameter begründen

👉 MUSS rein in eure Arbeit:

🌳 ntree

viele Bäume → stabil

z. B. 500

🌿 mtry

Zufallsvariablen pro Split

☐ minsplit / minbucket

verhindert kleine Gruppen

📏 mincriterion

Signifikanzniveau

bei euch:

$$p < 0.001$$

✓ 15. Varianz erklären

👉 deine Formel:

$$R^2 = 1 - (\text{MSE} / \text{Varianz})$$

✓ korrekt

👉 Interpretation:

wie gut erklärt Modell SHI

✓ 16. Vergleich mit ntree etc.

👉 ihr könnt machen:



ntreeR²1000.555000.62 ✓

☞ dann sagen:

➡ 500 besser

✓ 17. Ist Modell optimierbar?

☞ JA ✓

Optimierbar durch:

ntree

mtry

minsplit

mincriterion

☞ nennt man:

Hyperparameter-Tuning

✓ 18. Fachbegriffe aus Hobley (unbedingt verwenden!)

☞ diese Begriffe müsst ihr sagen:

✓ recursive partitioning

✓ node / root node / daughter node

✓ bootstrapping

✓ bagging

✓ feature importance

✓ out-of-bag error

✓ variance reduction

Deine Zusammenfassung ist inhaltlich schon ziemlich nah an dem, was man für eine Geoprojektarbeit zu Random Forest und SHI (Soil Health Index?) braucht. Ich würde einige Punkte wissenschaftlicher formulieren, damit sie direkt in die Arbeit übernommen werden können.

Ziel des Random-Forest-Modells

Ziel des Modells ist es, den SHI anhand von Umweltvariablen wie Klima, Geologie, Landnutzung und Topographie vorherzusagen. Dabei werden Entscheidungsbäume erzeugt, die durch rekursive Partitionierung (*recursive partitioning*) die Daten in möglichst homogene Gruppen aufteilen.

Optimierung der Splits

Bei jedem Split wird eine Variable und ein Schwellenwert gesucht, die zu einer möglichst starken Verringerung der Varianz innerhalb der entstehenden Gruppen führen (*variance reduction*). Für Regressionsprobleme entspricht dies der Minimierung des mittleren quadratischen Fehlers (MSE) bzw. des RMSE.

Die Grundidee lautet:

- geringe Varianz innerhalb der Gruppen
- große Unterschiede zwischen den Gruppen
- möglichst kleiner Vorhersagefehler

Bestimmung der Splitpunkte

Für jede potenzielle Splitvariable werden zahlreiche Schwellenwerte getestet, beispielsweise:

- Temperatur < 8 °C
- Temperatur < 10 °C
- Temperatur < 12 °C

Der Split mit der größten Fehlerreduktion wird ausgewählt.

Bootstrapping und Bagging

Random Forest basiert auf zwei zentralen Verfahren:

Bootstrapping



- Für jeden Baum wird eine Zufallsstichprobe mit Zurücklegen gezogen.
- Einzelne Beobachtungen können mehrfach vorkommen.

Bagging (Bootstrap Aggregating)

- Viele Bäume werden unabhängig voneinander trainiert.
- Die Vorhersagen werden anschließend gemittelt.
- Dadurch sinkt die Modellvarianz und die Robustheit steigt.

Zufällige Variablenauswahl (mtry)

Zusätzlich wird bei jedem Split nur eine zufällige Teilmenge der Prädiktoren betrachtet.

Parameter:

$$mtry \approx \sqrt{p}$$

mit

- p = Anzahl der Eingangsvariablen

Vorteile:

- verhindert Dominanz einzelner Variablen

- reduziert Korrelationen zwischen Bäumen
- verbessert die Generalisierungsfähigkeit

Out-of-Bag-Validierung

Etwa ein Drittel der Daten wird beim Bootstrapping nicht für einen bestimmten Baum verwendet.

Diese sogenannten **Out-of-Bag (OOB)-Daten** dienen zur:

- unabhängigen Modellbewertung
- Schätzung des Generalisierungsfehlers
- Berechnung des OOB-Errors

Dadurch ist keine zusätzliche Testmenge zwingend erforderlich.

Vermeidung von Overfitting

Zu viele Splits können zu einer Überanpassung führen.

Wichtige Parameter:

- **minsplit**: minimale Anzahl Beobachtungen für einen Split
- **minbucket**: minimale Anzahl Beobachtungen in einem Endknoten
- **maxdepth** (falls verwendet)
- **mincriterion** bei Conditional Inference Trees



Conditional Inference Trees

Im Gegensatz zu klassischen CART-Bäumen werden Splits nur durchgeführt, wenn ein statistisch signifikanter Zusammenhang vorliegt.

Beispielsweise:

$$p < 0.001$$

Vorteile:

- geringerer Selektionsbias
- weniger Overfitting
- fairere Variablenauswahl

Umgang mit korrelierten Variablen

Vor der Modellierung sollte die Korrelation zwischen den Prädiktoren untersucht werden.

Beispiel:

- Temperatur ↔ Höhe

Wenn

$$|r| > 0.8$$

kann man:

- eine Variable entfernen
- beide Variablen behalten, aber die Interpretation vorsichtig vornehmen

Modellgüte

Wichtige Kennzahlen:

RMSE

$$RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

Misst den mittleren Vorhersagefehler.

Bestimmtheitsmaß R^2

$$R^2 = 1 - \frac{MSE}{\text{Varianz}(y)}$$

Interpretation:

- $R^2 = 0$: keine Erklärungskraft
- $R^2 = 1$: perfekte Vorhersage



Hyperparameter-Tuning

Die Modellleistung kann durch Anpassung folgender Parameter verbessert werden:

Parameter	Funktion
ntree	Anzahl der Bäume
mtry	Variablen pro Split
minsplit	Mindestgröße für Splits
minbucket	Mindestgröße von Endknoten
mincriterion	Signifikanzniveau

Beispiel:

ntree	R^2
100	0.55
500	0.62
1000	0.62

Hier wäre 500 Bäume bereits ausreichend.

Feature Importance

Nach dem Training kann untersucht werden:

- welche Variablen den SHI am stärksten beeinflussen
- welche Interaktionen relevant sind
- welche Prozesse die räumliche Verteilung erklären

Wichtige Fachbegriffe für die Arbeit

Diese Begriffe solltest du unbedingt verwenden:

- Recursive Partitioning
- Root Node
- Node
- Daughter Node
- Bootstrapping
- Bagging
- Random Feature Selection
- Feature Importance
- Out-of-Bag Error
- Variance Reduction
- Hyperparameter Tuning
- Overfitting
- Generalization
- Conditional Inference Tree



Wenn du möchtest, kann ich daraus auch direkt einen **wissenschaftlichen Methodik-Abschnitt (1-2 Seiten für die Geoprojektarbeit)** formulieren und anhand deiner hochgeladenen CSV-Datei ein konkretes Python-Workflow-Beispiel mit **RandomForestRegressor** oder **cforest** erstellen.

wie entscheidet man die splitpunkte und den root node und branch node? und wie entscheidet man die minsplit, minbucket, maxdepth und mincriterion parameter um das overfitting zu vermeiden

Das ist eigentlich die zentrale Frage bei Entscheidungsbäumen und Random Forest. Wichtig ist: **Du entscheidest die Splitpunkte normalerweise nicht selbst. Das macht der Algorithmus.** Du entscheidest hauptsächlich die Parameter, die den Baum einschränken.

1. Wie wird der Root Node bestimmt?

Der Root Node ist der erste Split im gesamten Datensatz.

Angenommen ihr habt:

- Temperatur
- Niederschlag
- Höhe
- Landnutzung

und als Zielvariable SHI.

Der Algorithmus testet für jede Variable viele mögliche Splitpunkte:

Temperatur:

- Temp < 8
- Temp < 10
- Temp < 12
- ...

Höhe:

- Höhe < 300
- Höhe < 500
- Höhe < 700
- ...



Niederschlag:

- Niederschlag < 600
- Niederschlag < 700
- ...

Für jeden Split berechnet er die Varianzreduktion:

$$\Delta Var = Var_{vorher} - Var_{nachher}$$

Der Split mit der größten Varianzreduktion wird Root Node.

Beispiel:

Split	Varianzreduktion
Temp < 10	0.18
Höhe < 500	0.32
Regen < 700	0.24

→ Root Node:

Höhe < 500 m

weil die größte Varianzreduktion erreicht wird.

2. Wie entstehen Branches (Daughter Nodes)?

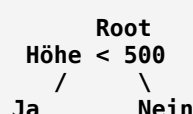
Nachdem der Root Node erstellt wurde, werden die beiden Gruppen getrennt betrachtet.

Beispiel:

Root Node:

Höhe < 500

ergibt:



Nun sucht der Baum in jeder Gruppe erneut den besten Split.

Linke Gruppe:

Temp < 11

Rechte Gruppe:

Regen < 800



Dadurch entstehen Daughter Nodes.

Das Verfahren nennt sich:

Recursive Partitioning

3. Wie wird ein Splitpunkt ausgewählt?

Bei Regression (SHI ist kontinuierlich) meist über Varianzreduktion bzw. MSE.

Idee:

Vor dem Split:

$$Var(Y) = 25$$

Nach dem Split:

Gruppe A:

$$Var = 8$$

Gruppe B:

$$Var = 10$$

gewichtete Varianz:

$$Var_{neu} = 9$$

Varianzreduktion:

$$25 - 9 = 16$$

Je größer die Reduktion, desto besser.

4. Warum entstehen Overfitting-Bäume?

Ohne Einschränkungen macht der Baum:

```
Höhe < 500
↓
Temp < 11
↓
Temp < 10.9
↓
Temp < 10.85
↓
...
```

Am Ende werden einzelne Beobachtungen isoliert.

Der Baum lernt Zufall statt Muster.



Parameter gegen Overfitting

minsplit

Mindestanzahl Beobachtungen, damit überhaupt gesplittet werden darf.

Beispiel:

```
minsplit = 20
```

Node enthält:

```
50 Punkte
```

→ Split erlaubt

Node enthält:

```
15 Punkte
```

→ Split verboten

Typische Werte

Stichprobe	minsplit
< 500	10–20
500–5000	20–50
> 5000	50–100

minbucket

Mindestanzahl Beobachtungen in jedem Endknoten.

Beispiel:

```
minbucket = 5
```

Ein Split erzeugt:

```
Links = 2
Rechts = 98
```

→ nicht erlaubt

weil links kleiner als 5 wäre.



Faustregel

$$\text{minbucket} \approx \frac{\text{minsplit}}{3}$$

oft:

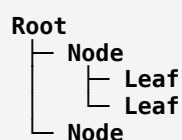
```
Python
minsplit = 30
minbucket = 10
```

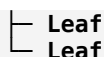
maxdepth

Maximale Baumtiefe.

Beispiel:

```
maxdepth = 3
```





Danach wird nicht weiter gesplittet.

Typische Werte

Datensatz	maxdepth
klein	3-8
mittel	5-15
groß	10-20

Bei Random Forest wird maxdepth oft gar nicht gesetzt und stattdessen über minsplit/minbucket reguliert.

mincriterion (Conditional Inference Trees)

Hier wird ein Split nur durchgeführt, wenn er statistisch signifikant ist.

In `partykit::cforest` gilt:

$$\text{mincriterion} = 1 - p$$

Beispiele:

p-Wert	mincriterion
0.05	0.95
0.01	0.99
0.001	0.999

Euer Wert:

```
R
mincriterion = 0.999
```

entspricht:

$$p < 0.001$$

Das ist sehr streng.

Wie entscheidet man die Parameter wissenschaftlich?

Nicht nach Gefühl.

Man verwendet:

OOB-Fehler

Random Forest liefert automatisch:

- OOB RMSE
- OOB R^2

Man testet verschiedene Kombinationen:

minsplit	minbucket	OOB R^2
10	3	0.58
20	5	0.63
30	10	0.67
50	15	0.62

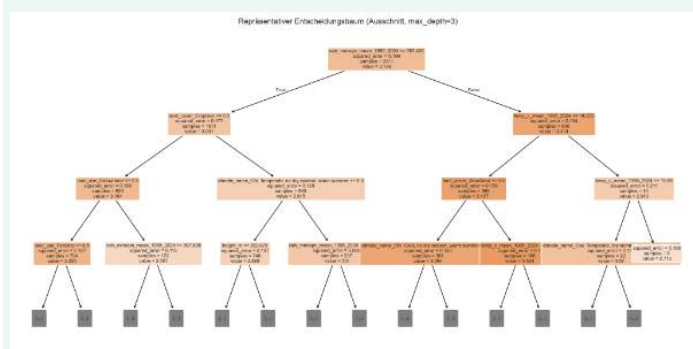
→ beste Kombination:

```
minsplit = 30
minbucket = 10
```

Für eure Arbeit könnte man schreiben

Die Hyperparameter wurden anhand des Out-of-Bag-Fehlers optimiert. Die Mindestanzahl an Beobachtungen pro Split (minsplit) sowie die Mindestanzahl an Beobachtungen in den Endknoten (minbucket) wurden schrittweise variiert. Die finale Parametrisierung wurde so gewählt, dass ein möglichst hoher OOB- R^2 -Wert bei gleichzeitig geringer Modellkomplexität erreicht wurde. Zusätzlich wurde mit einem Signifikanzniveau von $p < 0.001$ (mincriterion = 0.999) gearbeitet, um statistisch nicht belastbare Splits zu vermeiden und Overfitting zu reduzieren. □

Das ist eine Begründung, wie sie in Bachelor-, Master- und Geoprojektarbeiten typischerweise akzeptiert wird.



hobley_rf_erklaerungbeispiel_repaired.pdf

PDF